

Тестовое задание — Telegram Bot (TRON), часть 2

Цель

На этом этапе разрешается активно использовать нейросети и AI-инструменты.

Основная цель — посмотреть:

- как ты работаешь с AI как с инженерным инструментом;
- умеешь ли отлаживать AI-generated код;
- умеешь ли проверять информацию;
- способен ли доводить решение до рабочего состояния, даже если AI ошибается.

Важно:

в реальной работе AI помогает ускоряться, но почти никогда не выдаёт готовое идеальное решение с первого раза. Поэтому оценивается не “умение написать промпт”, а способность итеративно доводить результат до рабочего состояния.

Задание

Необходимо расширить существующего Telegram-бота.

Добавить команду:

`/analyze ADDRESS`

которая должна:

- анализировать последние транзакции адреса;
- пытаться определить тип активности адреса;
- формировать краткую текстовую сводку.

Минимальные требования

Бот должен пытаться определять хотя бы часть из следующего:

- обычный пользовательский кошелёк;

- контракт;
- биржевой адрес;
- высокая активность;
- большое количество мелких входящих переводов;
- подозрительная spam-активность;
- преимущественно входящие или исходящие транзакции.

Логика может быть эвристической и упрощённой.
Не требуется “настоящий AML-анализ”.

Использование AI

На этом этапе разрешается:

- генерировать код через AI;
- использовать Cursor / Copilot / ChatGPT / Claude и т.д.;
- рефакторить через AI;
- просить AI объяснять ошибки;
- использовать AI для проектирования архитектуры.

Важно

Будет оцениваться не “красота промпта”, а:

- способность выявлять ошибки AI;
- умение дебажить;
- понимание того, что именно делает код;
- умение проверять предположения;
- способность дробить проблему на части.

AI может:

- предлагать несуществующие API;

- использовать deprecated библиотеки;
- путать версии SDK;
- генерировать нерабочий async-код;
- предлагать небезопасные решения;
- hallucinate-ить методы и параметры.

Это нормальная часть процесса.

Дополнительные требования

AI_LOG.md

Необходимо создать файл AI_LOG.md.

В нём фиксировать:

- какие AI-инструменты использовались;
- для каких задач;
- какие ответы AI оказались полезными;
- какие были ошибочными;
- как ты понял, что AI ошибается;
- какие проблемы пришлось исправлять вручную.

Не нужно вставлять огромные логи переписки — важен краткий инженерный разбор процесса.

Commit History

Как и в первой части:

- работа должна вестись через Git;
- коммиты должны быть регулярными;
- желательно отражать этапы поиска и исправлений.

Что будет оцениваться

В первую очередь:

- способность работать с AI осознанно;
- умение отлаживать и проверять результат;
- самостоятельность мышления;
- инженерный подход;
- качество анализа проблем;
- способность доводить AI-generated код до рабочего состояния.

Важно

Разрешается:

- не реализовать часть идей;
- оставить ограничения;
- честно описать проблемы и недоработки.

Не требуется “идеальный production-ready сервис”.

Намного важнее прозрачность процесса и понимание происходящего.